

Binary Search Trees: Selection and Sequences

Design and Analysis of Algorithms

Brian C. Dean

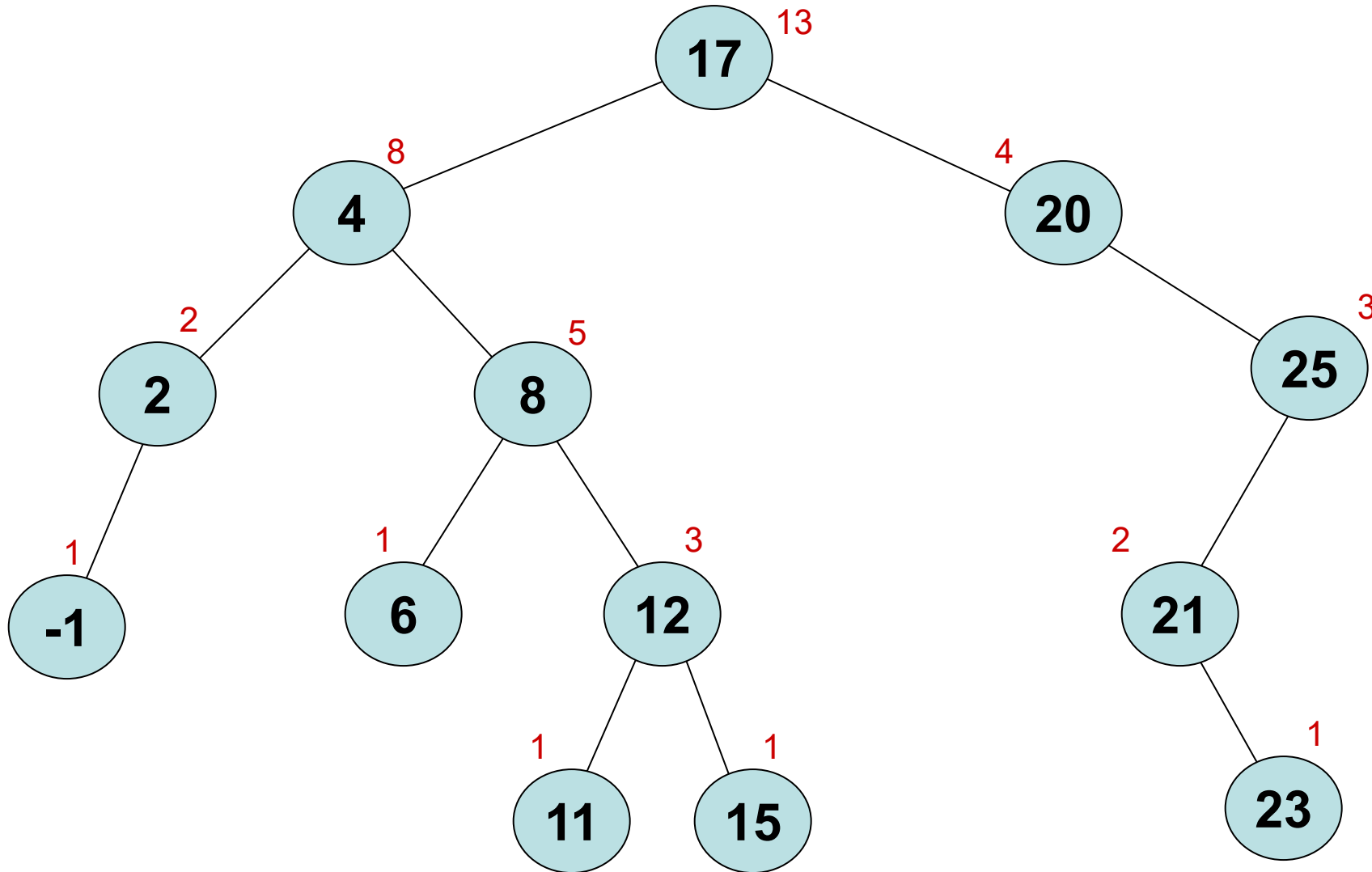
**School of Computing
Clemson University**



New Operations: Select and Rank

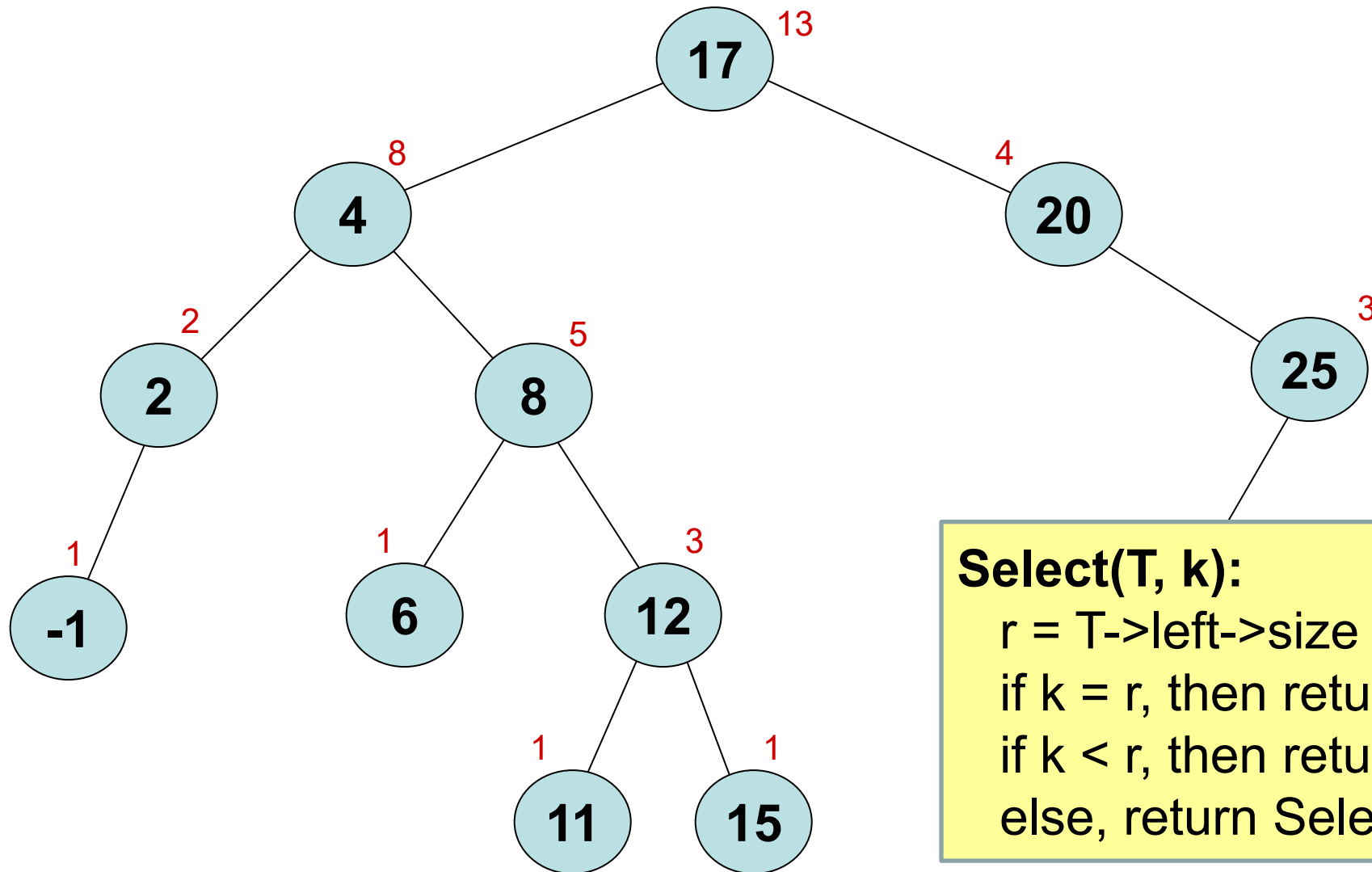
- *select*(k) : find the rank k element – the one that lives at index k within the inorder traversal of the tree.
 - $k = 0$ is the min
 - $k = n - 1$ is the max
 - $k = n / 2$ is the median.
- *rank*(e) : given a pointer to e , returns the number of elements with keys $\leq e$'s key.
 - (that is, e 's index within an inorder traversal)
- Simple application: sampling a random element by taking *select* (random number in $0 \dots n - 1$)

Need to Augment Nodes with Subtree Sizes



```
struct Node {  
    int key;  
    Node *left;  
    Node *right;  
    int size;  
};
```

Select in a BST is Directly Analogous to Selection via Quickselect



Select(T, k):

$r = T \rightarrow \text{left} \rightarrow \text{size}$ // $r = \text{rank of root}$

if $k = r$, then return T

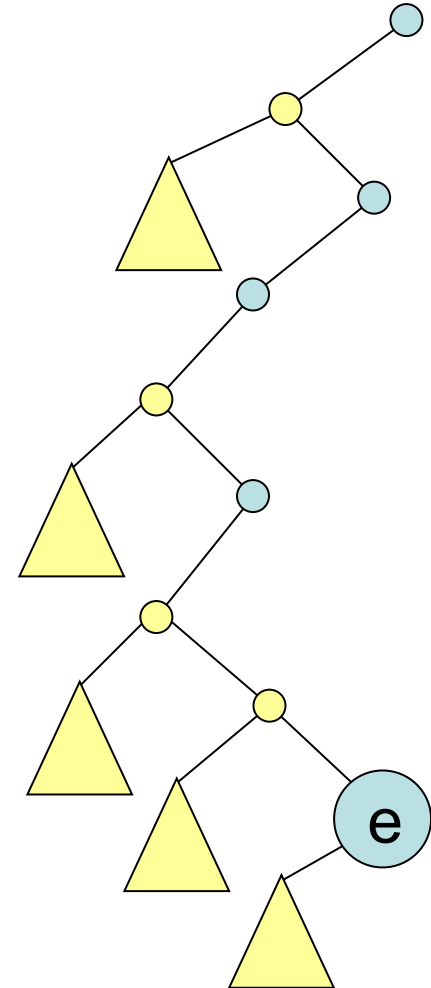
if $k < r$, then return $\text{Select}(T \rightarrow \text{left}, k)$

else, return $\text{Select}(T \rightarrow \text{right}, k - r - 1)$

Rank

Rank(e):

Add up the total # of yellow elements
as we walk from e up to the root.
(these are all the elements less than e)



Recall:

Array and Linked List Performance



- Retrieve or modify any element in $O(1)$ time, given its index.
- Insert or delete in middle of list: $O(N)$ time. 
- Insert or delete from ends: $O(1)$ time.



- Seek to any position in list: $O(N)$ time. 
- Then insert or delete element: $O(1)$ time.
- Insert or delete from ends: $O(1)$ time.

Recall:

Array and Linked List Performance



- Retrieve or modify any element in $O(1)$ time, given its index.
- Insert or delete in middle of list: $O(N)$ time. 
- Insert or delete from ends: $O(1)$ time.



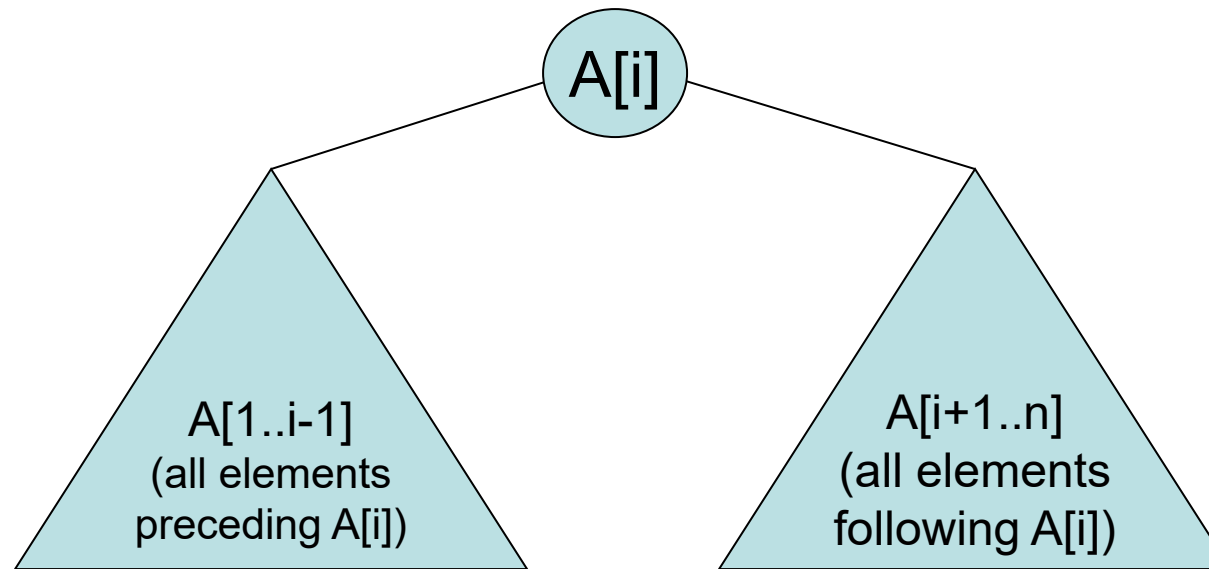
- Seek to any position in list: $O(N)$
- Then insert or delete element: $O(N)$
- Insert or delete from ends: $O(1)$ time

All of these operations run in $O(\log N)$ time using a balanced BST to implement a sequence!



Dynamic Sequences

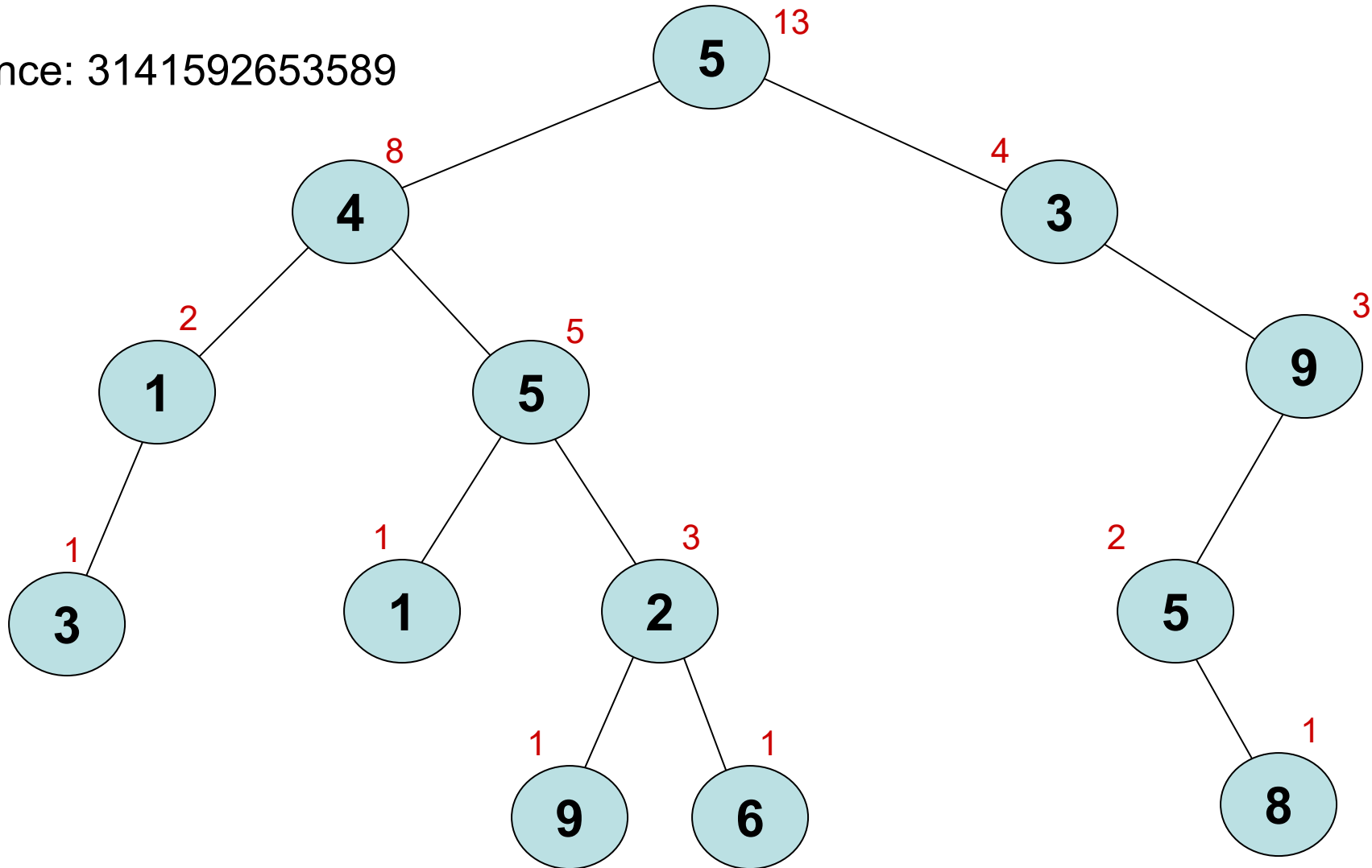
- The “BST property” is slightly different when we use a BST to encode a sequence $A[1..n]$:



- We no longer use *find*.
- Rather, we rely on rank-based access using *select* to insert, delete, and access elements based their index within the sequence.

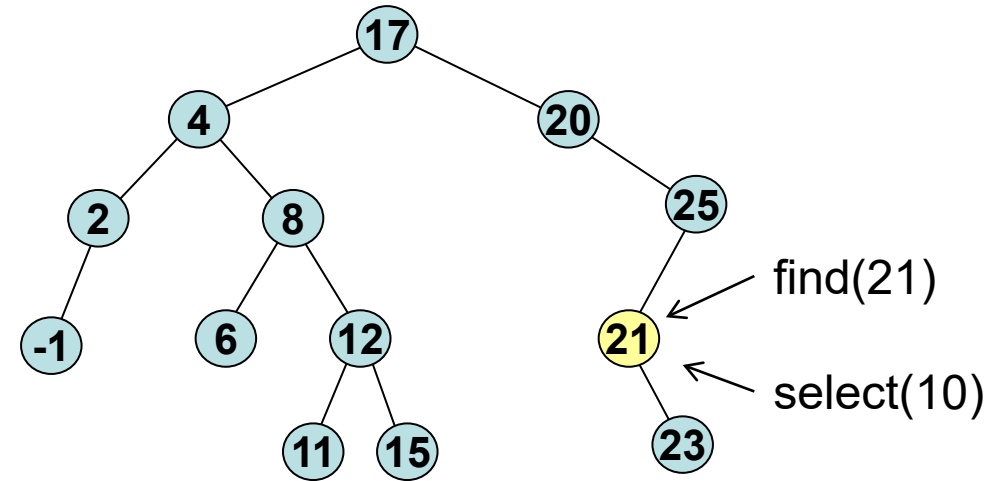
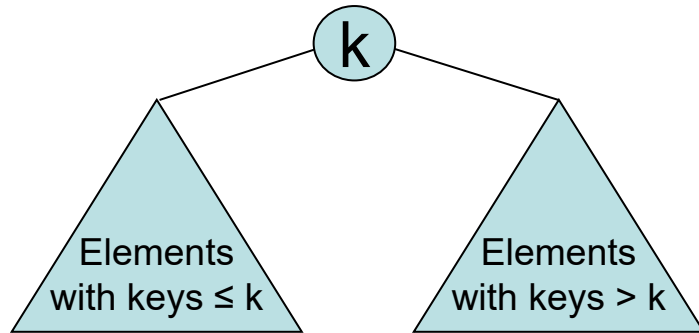
Dynamic Sequences : Example

Sequence: 3141592653589

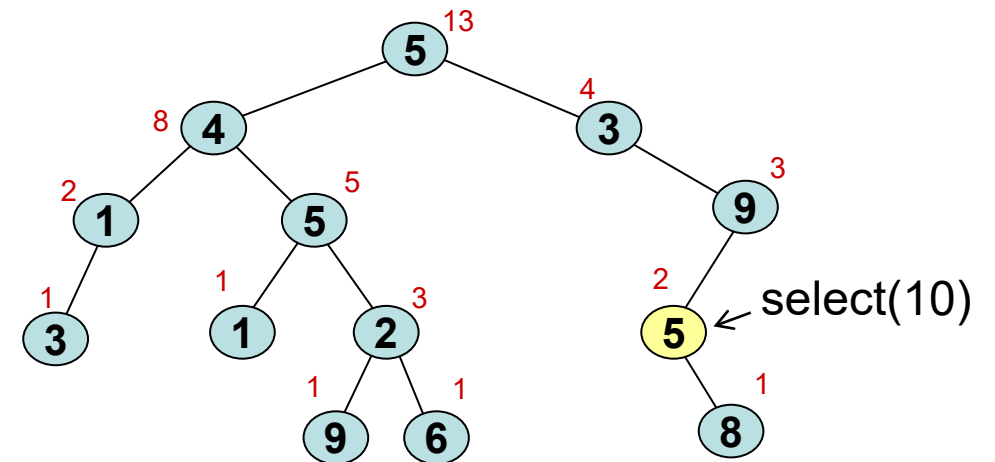
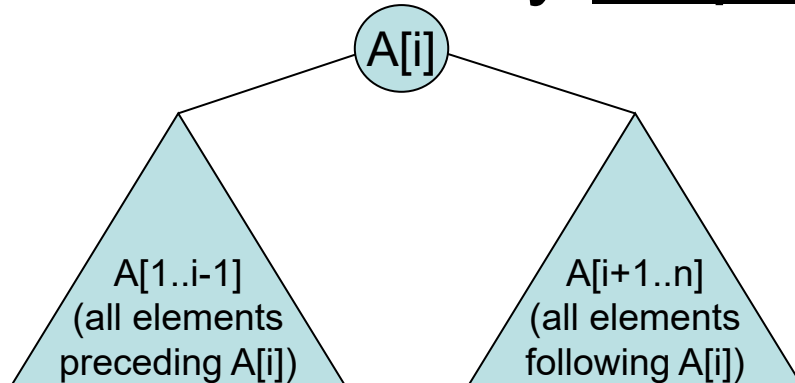


Binary Search Trees – Two Main Uses

- Implement a set or map



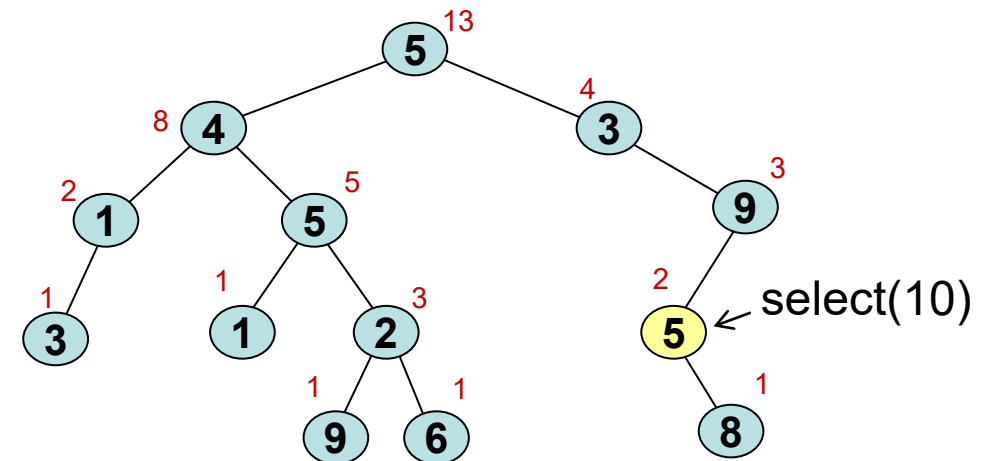
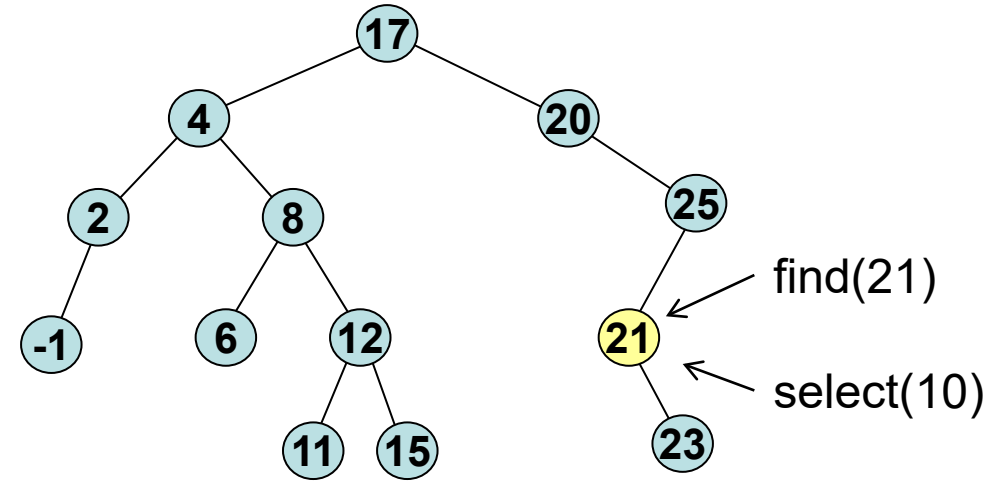
- Store an arbitrary sequence



Sequence: 3141592653589

Many Useful Operations that Make Sense in Either Case

- Implement a set or map
 - select, rank
 - pred, succ
 - min, max
 - split, join
- Store an arbitrary sequence
 - select(i) to access the ith element
 - pred, succ
 - min, max
 - split, join



Sequence: 3141592653589

Binary Search Trees and Sequences

- A binary search tree represents a sequence.
(in its inorder traversal)
- In the case of a set / dictionary, this sequence happens to be the sorted ordering of the elements we are storing.
- Strong connections with fundamental algorithms like binary search, quicksort, and quickselect.
- When used as a sequence, sometimes the term “order statistic tree” is used instead of “binary search tree”.
- In both cases, binary search trees provide a powerful way of interacting with this sequential data – the starting point for building a number of more advanced data structures (e.g., dynamic trees).